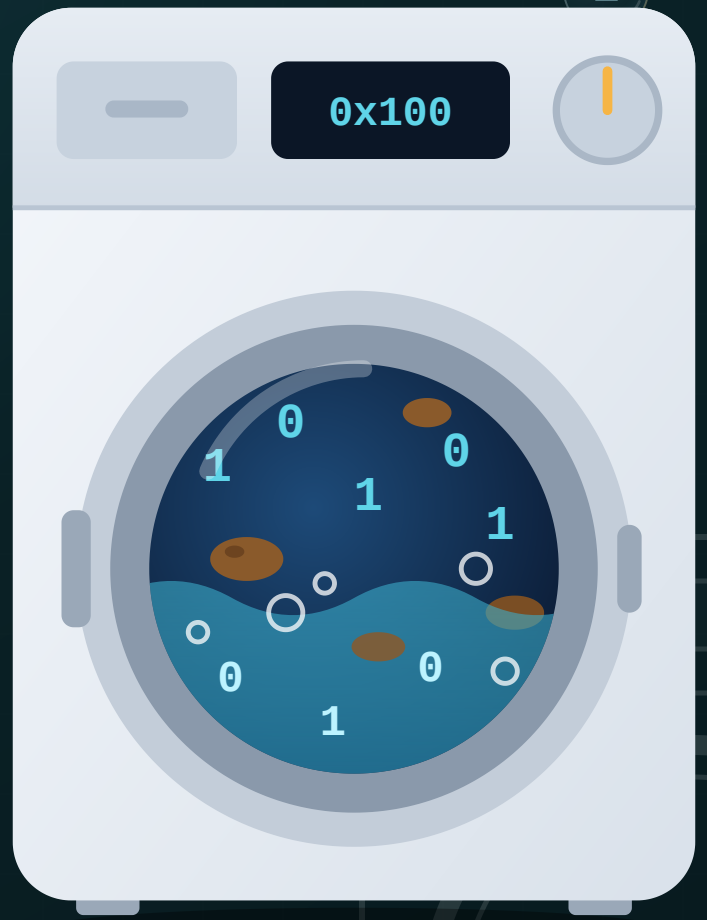
How clean is your code?

Dirty bits in Solidity

Doris Benda, Ph.D. · ChainSecurity

DeFi Security Summit 2026



Quiz: what is `z`?

```
uint256 x = 1;
uint8   y = 0x80;
y = y << 1;                    // 0x80 << 1 = 0x100 → truncated to 0

uint256 z;
assembly { z := add(x, y) }      // 1 + 0 = ?
```

A 1

B 257

C revert

No tricks: Solidity 0.8, no `unchecked`, one line of assembly.

A or B. It depends on a build flag.

z = 257

`forge build`
legacy pipeline

z = 1

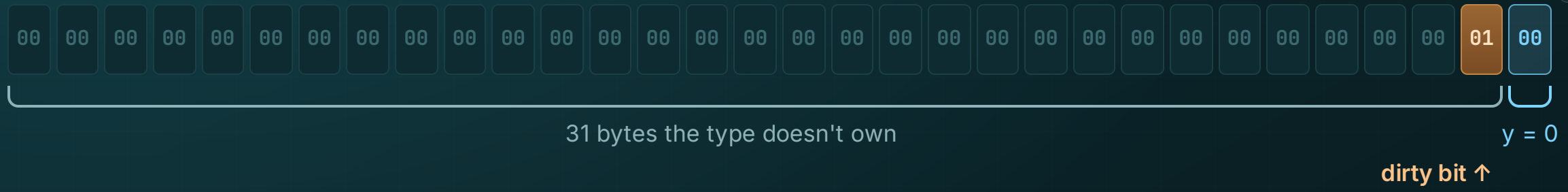
`forge build --via-ir`
via-IR pipeline
(IR = intermediate representation)

```
uint256 x = 1;
uint8   y = 0x80;
y = y << 1;                 // "y=0"

uint256 z;
assembly { z := add(x, y) }
// legacy: add(1, 256) = 257
// via-IR: add(1, 0)   = 1
```

248 bits nobody washes (legacy)

The stack word holding `uint8 y` after `y << 1`



> "You cannot make any assumptions about bits not part of the encoding of the type. Especially, do not assume them to be zero."
>
> Solidity docs · Inline Assembly

Clean means:

- `uintN` : left bits 0
- `intN` : left bits copy the sign bit
  `int8 5` = `0x00…05` · `int8 -5` = `0xff…fb`
- `bool` : exactly 0 or 1
- `address` : left 96 bits 0
- `bytesN` : right bits 0

Solidity reads **0**. Assembly reads **256**. Same 32 bytes.

Why would a compiler leave dirt?

Solidity and assembly both compile to **EVM opcodes** (**evm.codes**), which work on **full 256-bit words**. Washing costs gas, so Solidity washes only where the dirt would **change the result**.

Example: `uint8 y = 0` → clean word `0x000`, dirty word `0x100`

Dirt doesn't matter

`mul(y, 3)`

clean y	`0x…0000`	→ 0
dirty y	`0x…0300`	→ 0 ✓ **same**

Dirt changes the result

`eq(y, 0)`

clean y	`0x…0001`	→ true
dirty y	`0x…0000`	→ false ✗ **different**

Plain Solidity always washes for you.
But assembly never washes.



Two roommates - two cleaning habits

Legacy (default)

Leaves the dishes in the sink and washes right before guests arrive.

```
// Solidity    compiles to EVM opcodes
y = y << 1   → shl(1, y)              // 0x100 stays
y == 0       → eq(and(y, 0xff), 0)    // washed here
```

via-IR

Washes up right after cooking and never leaves dirty dishes in the sink.

```
// Solidity    compiles to EVM opcodes
y = y << 1   → and(shl(1, y), 0xff)   // 0x00
y == 0       → eq(y, 0)               // already clean
```

> Optimized output shown. Without `--optimize`, via-IR also masks before `eq`.

SOLIDITY CODE	RAW WORD, LEGACY	RAW WORD, VIA-IR
`uint8 y = 0x80; y = y << 1;`	256	0
`uint8 y = 0; y = ~y;`	$2^{256} - 1$	255
`uint8 y = uint8(uint256(0x1ff));`	511	255
`int8 y = 127; unchecked { y += 1; }`	+128	−128
`address y = address(uint160(uint256(hash)));`	full 256-bit hash	right 160 bits



Is the legacy pipeline still used?

`via_ir = false`

Foundry's default. A plain `forge build` is the **legacy** pipeline.

Hardhat: also off by default

61%

of unique contracts verified on Sourcify on one day were compiled with **legacy**.

Ethereum mainnet · 14 Oct 2026 · 206 unique bytecodes

35%

of Solidity developers **don't know** what the IR pipeline is.

Solidity Developer Survey 2025



When are ABI decoders v1 and v2 used?

COMPILED WITH	DECODER	
solc < 0.8	**v1**	masks dirty arguments
solc ≥ 0.8	**v2**	reverts on dirty arguments

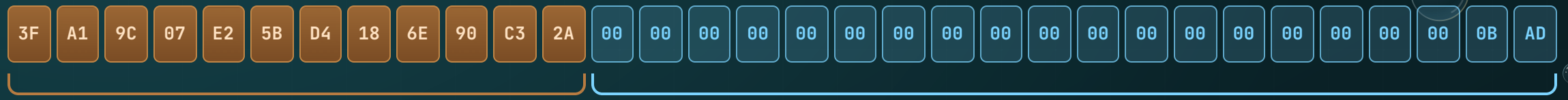
Exceptions:

- `pragma experimental ABIEncoderV2` enables v2 before 0.8
- `pragma abicoder v1` keeps v1 in 0.8
- `--via-ir` forces v2, overriding `pragma abicoder v1` if present

> "ABI coder v2 ... performs more sanity checks on the inputs. ... it can also make contract calls revert that did not revert with ABI coder v1."
>
> Solidity docs · 0.8.0 breaking changes



Two front doors: ABI decoder v1 vs v2




```
// Solmate SafeTransferLib.safeTransfer(token, to, amount), before June 2023
assembly {
    mstore(add(p, 4), to)                  // `to` copied raw: may be dirty
    ...
    call(gas(), token, 0, p, 68, 0, 32)    // token.transfer(to, amount)
}
```

Token compiled with ABI decoder v1

```
function transfer(address to, uint256 amt)
```

dirt masked silently → transfer works

WETH, USDT and most tokens

Token compiled with ABI decoder v2

```
function transfer(address to, uint256 amt)
```

dirt rejected → revert

UNI, COMP, FRAX (and CRV, Vyper)

Same dirty address: v1 **masked** it, v2 **rejected** it.

Two front doors: function vs fallback

```
// the function body, identical in both versions below
assembly {
    let to := calldataload(4)          // the address argument, raw
    ...                                // uses `to`
}
```

```
function withdraw(address, uint256)
    external { ... }
```

decoder v1	**dirty address gets through**
decoder v2	**revert: the decoder checks it**

```
fallback() external { ... }
```

decoder v1	**dirty address gets through**
decoder v2	**dirty address gets through**

> The decoder only checks **declared** input parameters.

Sources & sinks

A dirty-bit bug needs two things: someone who makes a mess, and someone who minds.



Who makes a mess · who minds

Sources of dirt

- **Solidity (legacy); for some types:** `<<` `>>` `~` `unchecked`, narrowing casts
- **Your assembly:** raw `calldataload` `sload` `mload`
- **Outsiders:** crafted calldata, return data
- **The compiler:** cleanup bugs

Sinks that care

- **Compare numbers:** `eq` `lt` `slt` `iszero`
- **Divide/shift numbers:** `div` `mod` `shr` `sar`
- **Hash:** `keccak256` for mapping keys, blocklists
- **Locate:** slot, index and offset maths

Full lists: chainsecurity.com/blog

> A source without a sink is not a bug. Many invalid reports have only one of the two.

Where dirty-bit bugs hide

SOURCE → SINK	LEGACY PIPELINE	VIA-IR PIPELINE
Solidity → assembly	**BUG RISK** assembly reads the dirt	**CLEANED** right after a dirt-producing step
assembly → assembly	**BUG RISK** nobody washes in between	**BUG RISK** nobody washes in between
assembly → Solidity	**CLEANED** right before a dirt-sensitive step	**MOSTLY CLEANED** except a storage array index → next slide



Oddity: the `require` that lies

```
uint256[4] arr;  arr[1] = 42;        // storage array
uint8 d;  assembly { d := 0x101 }    // "1" + a dirty bit
require(d < 4);
uint256 v = arr[d];
```

legacy

`require(d < 4)`	**sees 1 → passes**
`arr[d]`	**sees 1 → 42**

via-IR

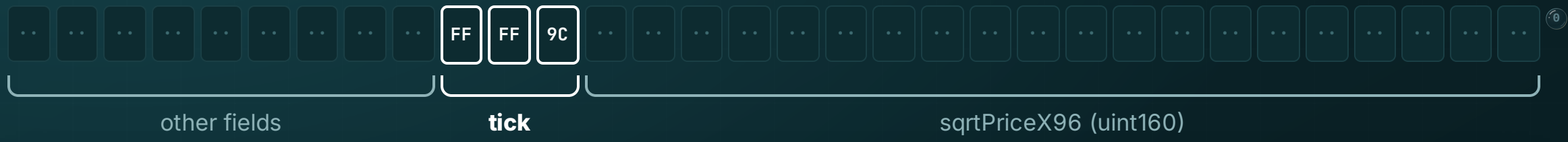
`require(d < 4)`	**sees 1 → passes**
`arr[d]`	**sees 257 → Panic 0x32**

In via-IR, the check and the array index read the **same variable** but get **different numbers**.



Wrong washing program: masking a signed tick

Uniswap V3 packs the price and the `int24 tick` into one storage slot (`slot0`). Tick −100:




```
int256 tick;  assembly { tick := shr(160, sload(slot0.slot)) }   // tick bytes now on the right
```

✘ Wrong program: mask

```
assembly { let wrongTick := and(tick, 0xffffff) }
```

wrongTick **16,777,116**

✔ Right program: sign-extend

```
assembly { let correctTick := signextend(2, tick) }
```

correctTick **−100**

A mask cleans unsigned values. Signed values need **signextend**, in legacy and via-IR.

Care labels for your assembly



`uint8`

```
and(x, 0xff)
```

mask the left



`address`

```
shr(96,shl(96,a))
```

do not tumble 96 bits



`int24`

```
signextend(2, x)
```

never mask: colours bleed



`bool`

```
iszero(iszero(b))
```

2 is not a boolean



`bytes4`

```
and(x, shl(224, 0xffffffff))
```

left-aligned: rinse the right

Before you start the next wash cycle:

1. Wash at both doors. Clean narrow values entering and leaving assembly (ensure no sensitive sink gets dirty values).

2. Fuzz with dirt. Solady's `Brutalizer` dirties inputs and memory.

3. Test what you ship. Legacy and via-IR clean differently. Fork/Test against the real tokens you integrate, as they might use different decoders.

4. Audit both ends. Dirt is only a bug once it reaches a sink.



Solidity does its own laundry. Assembly is a laundromat.

Doris Benda, Ph.D. · ChainSecurity

GitHub github.com/DOBEN

LinkedIn linkedin.com/in/dorisbenda

Blog
chainsecurity.com/blog

LinkedIn
in/dorisbenda

Slides
github.com/DOBEN



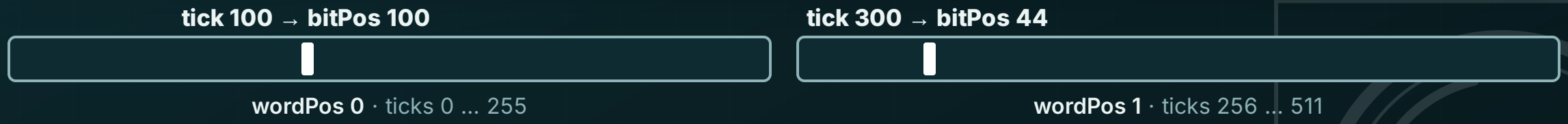
↪ DETOUR · DEFI BACKGROUND

TickBitmap in 30 seconds

Introduced by **Uniswap V3**, now used by many AMMs and order books.
Prices are split into **ticks** (`int24` , can be negative).
A bitmap stores **one bit per tick**: 1 = liquidity at that price.

A function maps a **tick** to its **position** (Uniswap V3, simplified):

```
function position(int24 tick) private pure returns (int16 wordPos, uint8 bitPos) {
    wordPos = int16(tick >> 8);     // which storage slot
    bitPos  = uint8(uint24(tick % 256));    // which bit · legacy: dirty for tick < 0
}
```



TickBitmap: one dirty `bitPos`, two readers

```
(, uint8 bitPos) = position(-100);   // should be 0x9c = 156
// legacy raw word: 0xffffffff…ff9c
```

Solidity

```
word ^= 1 << bitPos;
```

legacy	**bit 156 flipped ✔**
via-IR	**bit 156 flipped ✔**

Solidity washes `bitPos` before the shift · this is what Uniswap V3 does: safe

assembly

```
assembly { word := xor(word, shl(bitPos, 1)) }
```

legacy	**shl → 0: no flip**
via-IR	**bit 156 flipped ✔**

legacy shifts by $2^{256} - 100$ · via-IR washed `bitPos` when it was computed

Same value, two readers. In assembly, wash it:

`assembly { word := xor(word, shl(and(bitPos, 0xff), 1)) }` → bit 156 ✔ in both